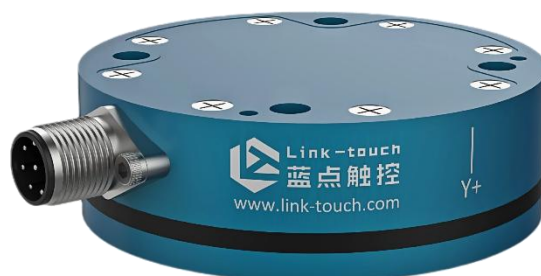




WRIST 系列六维力/力矩传感器



专注高精度力觉测量与控制
多维力觉传感器行业领跑者

Engineered Products for Sensor Productivity

版本号：1.5.6

前言

本文档中所包含内容归蓝点触控（北京）科技有限公司所有，任何公司及个人不得在未经我司事先书面同意的前提下使用和修改文档中的部分或全部内容。（此文档我司有权随时更改，另不再额外通知,声明不应视为我司承诺。）

本手册定期修订，高时效的反应产品的功能更新。我司对本文件中的任何错误或遗漏具有免责权力。

本文档版权归蓝点触控（北京）科技有限公司所有，公司保留所有权利。

考虑到我司 WRIST 系列产品是用于机器人或自动化控制领域，如果 WRIST 系列产品的组件或系统出现错误或故障，在错误或故障排除前我司不建议继续使用，以免导致伤害和危及操作人员生命的情况。

任何人和机构在任何潜在威胁生命的系统中使用或包含使用 WRIST 系列产品，必须事先得到我司的书面同意，在保证产品不构成直接或间接造成伤害或死亡的威胁的前提下授权使用。

注意：

在致电客户服务前请先仔细阅读产品说明书。在致电之前，请注意以下内容：

1. 传感器型号(如 ST-6-200-10-RS-1805012)
2. 准确和完整的对问题描述
3. 计算机和软件信息：操作系统，PC 类型，驱动程序，应用程序软件，以及与您的配置相关的其他信息。

如有可能，请在通话时靠近传感器。

如何联系我们：

地址：北京市海淀区创业中路 36 号 5 层 512 室

邮编：100085

销售或技术服务

电话：010-82782092 15301373062

邮箱：marketing@link-touch.com

网址：www.link-touch.com

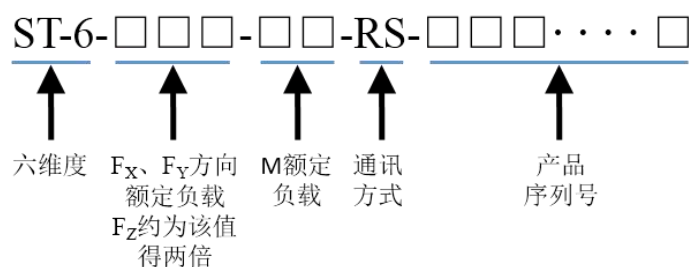
产品目录

前言	2
术语表	5
第一章 安全	6
1.1 声明	6
1.2 安全指导方针	6
1.3 安全预防措施	7
第二章 产品概述	8
第三章 安装说明	9
3.1 安装示意图	10
3.2 合理规划电缆路径	11
3.3 安装传感器	13
3.4 传感器拆除	14
3.5 传感器输出信号分配	14
3.6 精度检查	15
3.7 传感器额定力矩	15
第四章 操作说明	17
4.1 传感器使用环境	17
4.2 LED 功能说明	17
4.3 RS422 通讯	17
4.3.1 通信内容	18
4.3.2 通信协议	18
4.4 EtherCAT 通讯	19
4.5 以太网 UDP 通讯	19
4.5.1 传感器信息	19
4.5.2 通信协议	20
4.6 Modbus TCP 通讯	21
4.6.1 配置信息	21
4.6.2 功能说明	21
4.6.3 通信协议	21
第五章 维护	23

5.1 定期检修	23
5.2 定期校准	23
5.3 故障诊断	23
第六章 产品规格	24
6.1 存储和使用条件	24
6.2 产品型号参数	24
第七章 图纸	25
第八章 产品应用示例	26
附录 1 RS422 通讯示例代码	27
1、C++代码实现	27
附录 2 以太网 UDP 通讯示例代码	28
1、Visual Studio 例程	28
2、Linux 例程	30
3、网络调试助手用例	32
附录 3 Modbus TCP 通讯示例代码	36
1、传感器系统信息写入	36
2、传感器系统信息读取	37

术语表

名称	定义
精度	反映测量结果与真值接近程度的量，称为精度，它与误差的大小相对应，因此可用误差大小来表示精度的高低，误差小则精度高，误差大则精度低
分辨率	系统可以测量负荷的最小变化量，这通常比精度小的多
校准	在规定条件下，为确定传感器的功能准确，使用已知质量的被测量物体进行校准
产品认证证书	声明设备已经按照国家标准进行了测试，符合应用领域内的国家标准要求
复合加载	多轴复合加载
坐标系	设置坐标原点和运动计算参考基准
自由度	六个自由度方向上的输入
RS422 总线	RS-422 串口通讯总线，支持一对多的数据差分数据传输，抗噪能力强，传输距离远
RS422 通讯协议	RS-422 中数据传送控制的一种约定
力/力矩	施加在物体上的推力或拉力/使物体产生转动的力与作用轴或点的矩。
力和力矩传感器	将力和力矩信号转换为数字信号输出的装置。
量程	传感器正常使用时，可支持力和力矩测量的范围。
全量程测量误差	如果传感器的测量范围为 100 牛顿，传感器精度到 1 牛顿以内，传感器的全量程误差为 1%(1%=0.01=1N/100N)。
迟滞	由先前加载残余效应导致的测量误差
IP65	电气设备外壳对异物侵入的防护等级：6-完全防止粉尘进入，5-任何角度低压喷射无影响。
超载	超量程的加载导致传感器承载超过量程
六维输入	$F_x, F_y, F_z, M_x, M_y, M_z$ 。
工具端	传感器用于安装用户自定义的工具或工具转接件的一侧
固定端	传感器用于固定传感器自身的一侧
UDINT	32 位无符号的整数数据类型。
UINT	16 位无符号的整数数据类型。
USINT	8 位无符号的整数数据类型。
DINT	32 位有符号的整数数据类型。



第一章 安全

本章节主要声明产品的通用性安全操作规范、适用于本产品的安全预防措施。具体的安全操作步骤在手册的相关章节有详细的阐述。

1.1 声明

本章节中包含的警示说明是针对本手册所涵盖的 WRIST 系列产品。用户应注意所有机器人系统组件制造商和安装过程中涉及的其他部件制造商的安全操作说明。



危险：如果不遵守安全操作规范将导致死亡或严重伤害。该通知提供了有关危险情况的性质、未避免危险的后果以及避免这种情况的方法的信息。



警告：如果不遵守安全操作规范将导致死亡或严重伤害。该通知提供了有关危险情况的性质、未避免危险的后果以及避免这种情况的方法的信息。



当心：如果不遵守，可能导致中度伤害或对设备造成损害。该通知提供了有关危险情况的性质、未避免危险的后果以及避免这种情况的方法的信息。

注意：如果不遵照产品的维护、操作、安装或设置的特定说明执行，可能会导致设备损坏。特定说明包括但不限于特定的、良好的操作习惯或维护技巧。

1.2 安全指导方针

用户应确认所选传感器的额定负载和力矩不超过工作过程中预期的最大负载和力矩。由于静态载荷小于来自机器加速过程中产生的动态载荷，所以要注意机器的动态载荷。

1.3 安全预防措施



警告：当电路(如电源、水和空气)导通时，对传感器进行维护或修理可能会导致传感器的严重损伤。用户应根据用户的安全操作规范，确认所有通电电路是否通电。



当心：随意的修改或拆卸传感器可能会导致传感器组件的损坏和保修无效（脱保）。用户应使用我司提供的安装组件安装，并基于操作手册规范开展调试工作。



当心：传感器的破损会对传感器件内设备造成损害。用户应避免传感器的的外壳损伤。



当心：传感器单轴加载过载，复合加载过载，都将对传感器本身造成不可弥补的损坏。



当心：不要随意碰触 IP65 密封装置，这可能会导致传感器失灵。



当心：用户应避免静电放电对传感器造成的损坏。在处理与传感器相连的设备或电缆时，确保遵循正确的接地方式。如果不遵循，可能会损坏传感器。



当心：传感器在运输过程中应避免超出其承受范围的冲击和冲击载荷，因为冲击会损害传感器的内部结构，导致其性能劣化的损坏。



当心：安装方式和组件选择需合理，使用超规范的紧固件，会破坏传感器本体，损坏传感器内部电子元件，导致传感器失效和脱保。

第二章 产品概述

WRIST 系列传感器测量工具端三个互相垂直的方向的力 (F_x , F_y , F_z) 和力矩 (M_x , M_y , M_z)，并通过通讯电缆传输到用户设备实时处理。传感器固定端与用户机机构的执行端相连，工具端连接用户自定义工具。

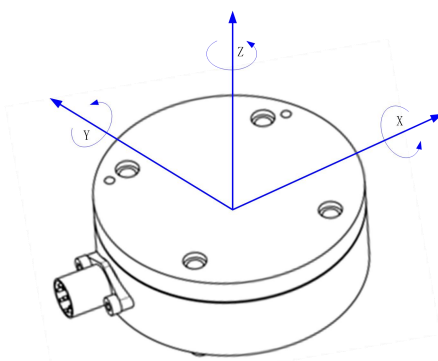


图 2.1 传感器坐标定义

WRIST 系列传感器外观结构如图 2.1 所示，具有以下特点：

- 1、力和力矩高精度解耦输出
- 2、适合流水线机器人的尺寸和安装结构
- 3、温度自动补偿技术
- 4、极高的分辨率
- 5、高响应带宽。

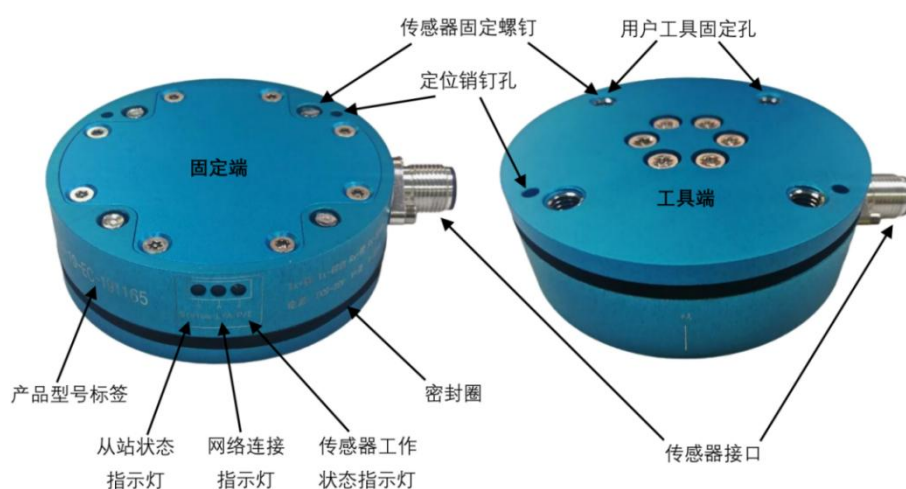


图 2.2 传感器外观结构

第三章 安装说明



警告：当电路(如电源、水和空气)导通时，对传感器进行维护或修理可能会导致传感器的严重损伤。用户应根据用户的安全操作规范，确认所有通电电路是否通电。



当心：随意的修改或拆卸传感器可能会导致传感器组件的损坏和保修无效（脱保）。用户应使用我司提供的安装组件安装，并基于操作手册规范开展调试工作。



当心：安装方式和组件选择需合理，使用超规范的紧固件，会破坏传感器本体，损坏传感器内部电子元件，导致传感器失效和脱保。



当心：传感器的破损会对传感器件内设备造成损害。用户应避免传感器的的外壳损伤。



当心：传感器单轴加载过载，符合加载过载，都将对传感器本身造成不可弥补的损坏。



当心：不要随意碰触 ip65 密封装置，这可能会导致传感器失灵。



当心：用户应避免静电放电对传感器造成的损坏。在处理与传感器相连的设备或电缆时，确保遵循正确的接地方式。如果不遵循，可能会损坏传感器。

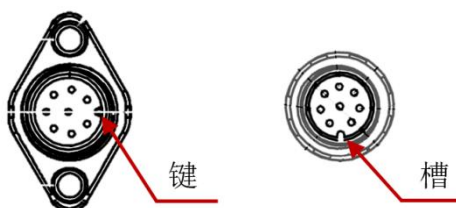


当心：传感器在运输过程中应避免超出其承受范围的冲击和冲击载荷，因为冲击会损害传感器的内部结构，导致其性能劣化。



当心：用于紧固件的螺纹胶不能重复使用，否则会导致紧固件松动，对设备造成损坏，重复使用紧固件时，应始终使用新的螺纹胶。

当心：在安装过程中应对齐传感器和电缆连接器上的键槽，卡合安装，切忌不要对传感器和电缆连接器施加过大的力，否则会对连接器造成损坏。



3.1 安装示意图

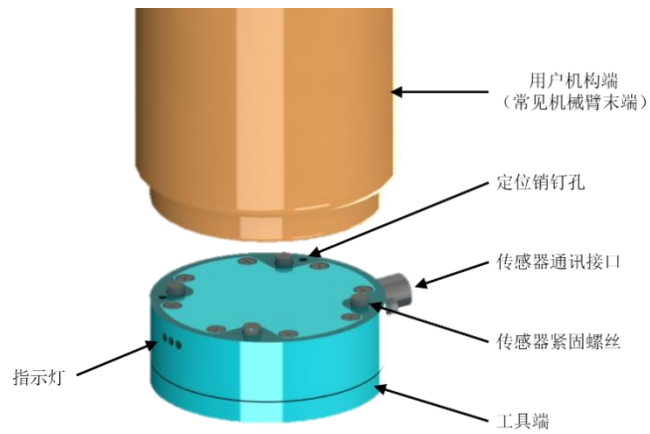


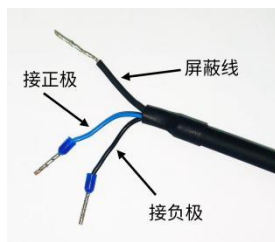
图 3.1 传感器安装示意图



图 3.2 线缆安装示意图 (适配 RS422 通讯)



图 3.3 线缆安装示意图（适配 EtherCAT/Ethernet 通讯）



蓝色线接正极，黑色线接负极，屏蔽线接地

图 3.4 电源线缆定义

3.2 合理规划电缆路径

电缆的布线路径和弯曲半径取决于用户的应用场景。不同于静态工况下的电缆应用（电缆处于静态的约束下），在电缆的动态应用中，电缆在外力的牵引作用下反复运动，电缆本身不仅承受牵引力的拉伸、扭转作用，同时还受到固定约束的挤压，使电缆产生复合变形，损伤电缆(包括机械外伤和内部绝缘损伤)，给运行留下隐患。

合理的约束电缆运动、规划电缆路径,有利与数据的良好通讯和延长连接器的使用寿命。

注意：最大可支持通讯电缆长度为 30m。

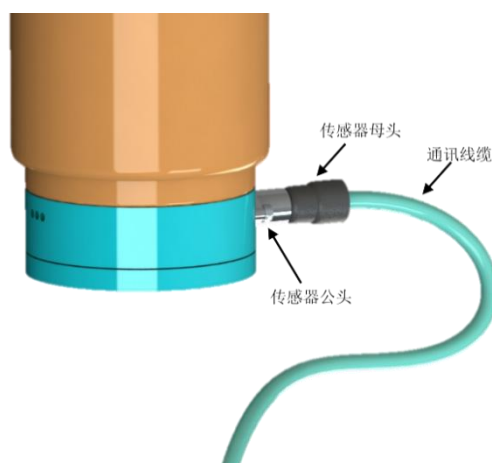


图 3.5 接线示意图



当心：布线时电缆的弯曲半径不应小于下表中规定的最小弯曲半径，过小的弯曲半径会导致电缆因机器人重复运动的疲劳而失效。



当心：避免连接器的输出电缆与反复运动的机构件直接相连，造成电缆对连接器的反复拉扯。实际操作时，用户应对靠近连接器的电缆额外增加一个约束固定，使机构件的重复运动与电缆连接器隔离。



当心：电缆必须经得起机构间的反复运动而不发生故障。电缆，尤其是连接到传感器连接器上的电缆，必须进行合理的规划约束，以避免应力反复、急剧弯曲导致的断路。不正确的路径规划可能会导致关键线路功能不良、人员受伤或设备损坏。因不正确的电缆规划而导致传感器损坏的将取消保修。如果应用环境导致电缆摩擦，可使用一个柔软的电缆保护套来包裹。

表 3.1 电缆弯曲半径及动态扭转角

电缆型号	功能描述	电缆直径 D(mm)	静态弯曲半径 (室 温)(mm)	动态弯曲半径 (室 温)(mm)	电缆动态扭转角度 /单位长度
S/FTP CAT.6	数据传输 及供电	6.5	8*D	10*D	180°/m

注意：

- 1、温度会影响电缆的柔韧性，建议在较低温度下布线时合理的增加电缆的最小动态弯曲半径。
- 2、切勿过紧的约束电缆，导致电缆内部线路的损坏。传感器接口处的电缆在机构整个的运动范围内不受应力、拉力、扭结、切割或其他损坏。如果应用导致电缆磨损，可使用一个柔软的电缆保护套来包裹电缆的外层材料。

3.3 安装传感器

定位要求：参考第七章图纸

工具要求：4.0mm 内六角扳手

补充要求：安装环境干净整洁、Loctite 242 螺纹胶水

安装步骤：

1. 确保传感器固定端面和机构的安装面清洁，无碎片。
2. 使用 4 毫米内六角扳手将传感器通过紧固件连接到机构表面。拧紧至 $6\text{N} \cdot \text{m}$ 。
 - a、螺纹的最小螺纹啮合长度为 4.5mm，最大螺纹啮合长度 6.5mm。
 - b、如果传感器要在高振动环境中使用，可使用螺纹胶加固。
3. 传感器安装在机器人上后，即可在用户端安装用户自定义的工具。
 - a、螺纹的最小螺纹啮合长度为 4.0mm，最大螺纹啮合长度 5.5mm。
 - b、如果传感器用于高振动环境，紧固件应采用螺纹胶，确保工具与传感器工具端紧密连接。
4. 将电缆 S/FTP CAT.6 连接到传感器和用户程序端。
 - a. 当传感器供电时，LED 自检。
5. 安装完成后，校准传感器精度，校准通过即可正常使用。

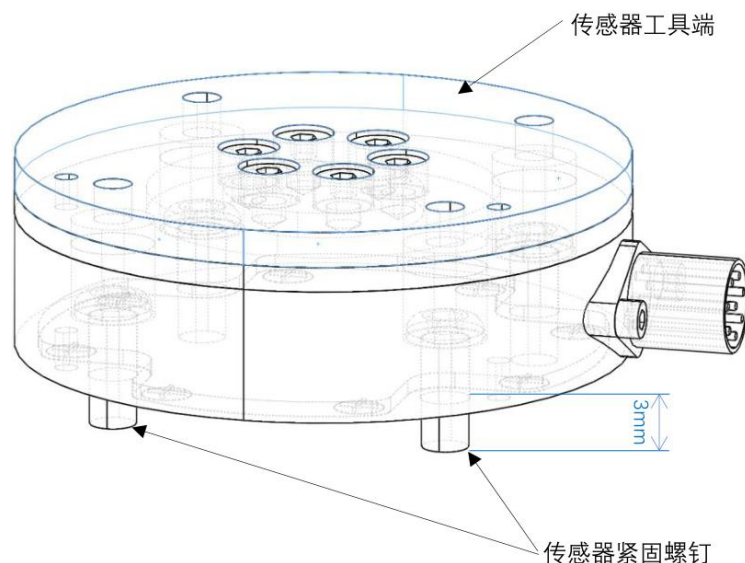


图 3.6 传感器透视图



如图 3.6 所示，为了在运输途中保护传感器，四个紧固螺钉不会完全收入传感器内部，而是会有大约 3mm 的尾部螺纹暴露在外部。安装时如果操作方法不正确，会使紧固螺钉头部顶到工具端，严重时损坏传感器。

使用建议：将四个紧固螺钉拧入螺纹孔时，切勿直接将某个螺钉拧到底，首先应将四个螺钉依次拧入至少 3mm（通常来说需将螺钉至少旋转 5 圈，如果旋转过程中突然感觉到轻微的卡顿，需立即停止旋转此螺钉，此时应旋转下一个螺钉），然后才能将螺钉拧紧至 $6\text{N} \cdot \text{m}$ 。

3.4 传感器拆除

工具要求：4.0mm 内六角扳手

- 1、关闭所有电源。
- 2、从传感器连接器上拆卸电缆 S/FTP CAT.6。
- 3、从传感器工具端上拆卸用户自定义工具。
- 4、手扶传感器，使用内六角扳手松开固定螺钉，拆卸传感器。

注意：

确保电缆护套务必正确接地。电缆屏蔽不当会导致通信错误和传感器失效、传感器噪声过大。



如图 3.6 所示，为了在运输途中保护传感器，四个紧固螺钉不会完全收入传感器内部，而是会有大约 3mm 的尾部螺纹暴露在外部。安装时如果操作方法不正确，会使紧固螺钉头部顶到工具端，严重时损坏传感器。

使用建议：将四个紧固螺钉拧出螺纹孔时，切勿直接将某个螺钉完全拧出，首先应将四个螺钉依次拧出至少 3mm（通常来说需将螺钉至少旋转 5 圈，如果旋转过程中突然感觉到轻微的卡顿，需立即停止旋转此螺钉，此时应旋转下一个螺钉），然后才能将螺钉全部拧出。

3.5 传感器输出信号分配

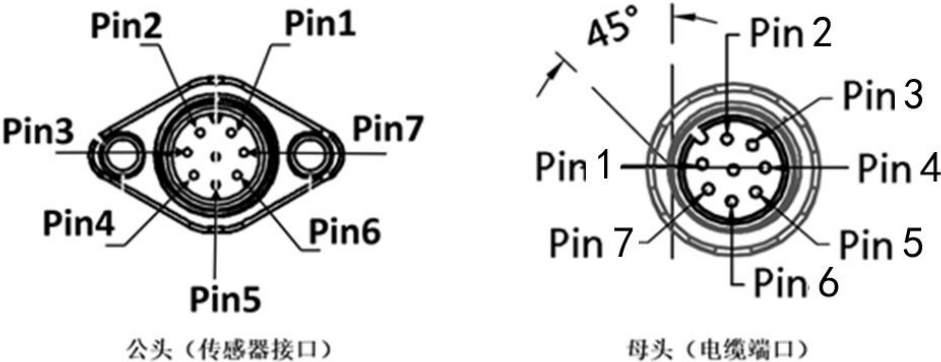


图 3.7 接口 Pin 脚示意图

下表详细说明了电缆 S/FTP CAT.6 内部信号分配及相应的连接器 pin 脚编号。

表 3.2 通讯电缆信号分配

S/FTP CAT.6 信号分配及接线指引			
Pin 脚	信号	Pin 脚	信号
1	V-	5	NA
2	Tx-	6	Rx-
3	Tx+	7	V+
4	Rx+	8	NA

3.6 精度检查

在完成传感器初次安装和周期性维护后，需要对传感器的精度进行检查。

1、在传感器的工具端固定一个已知质量的质量块。

2、移动机构端使的传感器处于以下的位置上,记录传感器在每个位置上的 F_X ， F_Y ， F_Z 的稳定输出：

点一：Z 向朝上；点二：X 向朝上；点三：Y 向朝上；

点四：X 向朝下；点五：Y 向朝下；点六：Z 向朝下

3、求得 6 个点三个方向的平均值：

$$F_{X.average} = (F_{x1} + F_{x2} + \dots + F_{x6}) / 6$$

$$F_{Y.average} = (F_{y1} + F_{y2} + \dots + F_{y6}) / 6$$

$$F_{Z.average} = (F_{z1} + F_{z2} + \dots + F_{z6}) / 6$$

4、对每一个点，完成如下计算：

$$F_X = F_{Xn} - F_{X.average}$$

$$F_Y = F_{Yn} - F_{Y.average}$$

$$F_Z = F_{Zn} - F_{Z.average}$$

$$G = \sqrt{F_X^2 + F_Y^2 + F_Z^2}$$

5、 $S = \text{Max } G - \text{Min } G$

$$S < 36N$$

当每次进行精度校准时，S 的取值都应该在 36N 之内。

3.7 传感器额定力矩



当实际力矩超过规定的额定力矩（即 M_x 、 M_y 和 M_z 值）时，有可能会损坏传感器。力矩值是力和力臂的乘积，当两者中任何一个的值比较大时，都会导致力矩值过大。

使用建议：

如图 3.8 所示，当 5kg 负载的重心到传感器的距离为 0.2m 时，施加到传感器的最大力矩可近似计算为 $5\text{kg} \times 10\text{N/kg} \times 0.2\text{m} = 10\text{ N}\cdot\text{m}$ 。如果这个负载再长一些（即重心到传感器的距离再远一些），或者工具受到外力（比如用该工具打磨工件），最大力矩就会超过 $10\text{ N}\cdot\text{m}$ ，可能会损坏额定力矩为 $10\text{ N}\cdot\text{m}$ 的传感器。

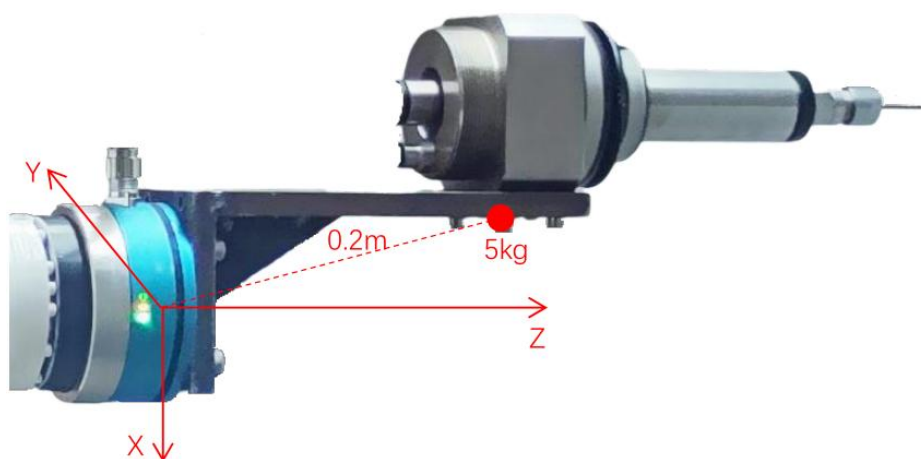


图 3.8 负载力矩示意图

如图 3.9 所示，当 10kg 负载的重心到传感器的距离为 0.1m 时，施加到传感器的最大力矩可近似计算为 $10\text{kg} \times 10\text{N/kg} \times 0.1\text{m} = 10\text{ N}\cdot\text{m}$ 。如果这个负载再长一些（即重心到传感器的距离再远一些），或者工具受到外力（比如用该工具夹起工件），最大力矩就会超过 $10\text{ N}\cdot\text{m}$ ，可能会损坏额定力矩为 $10\text{ N}\cdot\text{m}$ 的传感器。

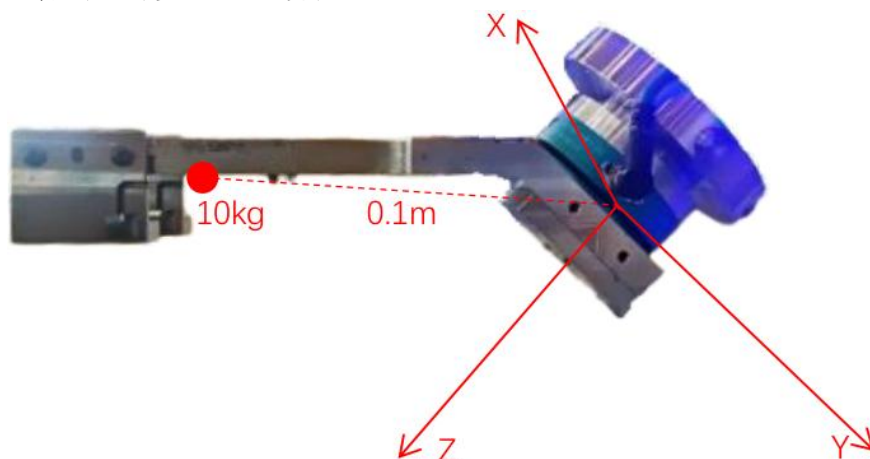


图 3.9 负载力矩示意图

第四章 操作说明

本章节将告知用户如何通过程序来操作传感器，与传感器的交互要求熟悉 RS422 串口、EtherCAT 总线、以太网 UDP 或者 Modbus TCP 通信协议。

4.1 传感器使用环境

为保障传感器的使用寿命和精度，传感器的使用环境必须和传感器的 IP 等级相符。本产品防护等级为 IP65。

注意：传感器外壳的损坏可能使湿气或水进入密封的传感器当中，检查确保电缆外套处于良好状态防止设备损坏。

注意：强磁场，交变磁场会对传感器的性能产生干扰，例如由磁共振成像(MRI)机产生的电磁场。

当施加电源电压时，传感器启动。传感器从启动后至其内部状态变稳定为止的数秒内是没有响应的。请在 5 秒或更长时间后，再向传感器输入命令。

在传感器刚启动后，传感器输出值可能由于内部温度变化而波动。为了稳定传感器的输出，请预热 10 -30 分钟后使用。

设备采样率：1000HZ

低通滤波：500HZ

4.2 LED 功能说明

网络连接指示灯（L/A）：以太网通讯线缆连接时指示灯为绿色快闪状态，断开时指示灯为常灭状态。

从站状态指示灯（Status）：对于 EtherCAT 通讯，从站处于 Init/Bootstrap 状态时指示灯为常灭状态，处于 Pre-Op 状态时指示灯为黄色快闪状态，处于 Safe-Op 状态时指示灯为黄色慢闪状态，处于 Op 状态时指示灯为黄色常亮状态。对于其它类型的通讯，指示灯为常灭状态。

传感器状态指示灯（P/E）：传感器正常工作时指示灯为黄绿色常亮状态，传感器断电时指示灯为常灭状态。

4.3 RS422 通讯

数据采集上位机与采集电路之间采用标准 RS422 通信方式进行信息交互，波特率为 921600bps，8 位数据位，1 位停止位，无校验位。

上电为默认连续发送状态。

4.3.1 通信内容

通信内容	发送方	接收方	输出频率
采集数据	传感器	上位机	1000Hz
数据停止发送命令	上位机	传感器	触发
数据开始发送命令	上位机	传感器	触发

4.3.2 通信协议

1、采集数据

字节序号	字节内容	备注
1	0xA5	字头
2~5	时间	float 型，低字节在前
6~9	1 通道数据	float 型，低字节在前
10~13	2 通道数据	float 型，低字节在前
14~17	3 通道数据	float 型，低字节在前
18~21	4 通道数据	float 型，低字节在前
22~25	5 通道数据	float 型，低字节在前
26~29	6 通道数据	float 型，低字节在前
30~33	温度	float 型，低字节在前
34~35	备用	
36	0x0D	字尾

2、数据停止发送命令

字节序号	字节内容	备注
1	0xC0	字头
2	0xF8	
3	0xF8	
4	0xC0	字尾

3、数据开始发送命令

字节序号	字节内容	备注
1	0xC0	字头
2	0xF7	
3	0xF7	
4	0xC0	字尾

4.4 EtherCAT 通讯

PDO 接口

TxPDO

Subindex	Name	Type	Description
0x01	Fx	DINT	X 轴的力, 10000 对应 1N
0x02	Fy	DINT	Y 轴的力, 10000 对应 1N
0x03	Fz	DINT	Z 轴的力, 10000 对应 1N
0x04	Mx	DINT	Mx 的力矩, 10000 对应 1N.m
0x05	My	DINT	My 的力矩, 10000 对应 1N.m
0x06	Mz	DINT	Mz 的力矩, 10000 对应 1N.m
0x08	Sample Counter	UDINT	运行时间, 1 对应 1ms, 可根据该数值判断当前接收到的数据和上一次接收到的数据是否同一帧
0x09	Temper	DINT	温度, 10 对应 1℃

RxPDO

Subindex	Name	Type	Description
0x01	Control Code	DINT	第 0 位上升沿时传感器数据清零

4.5 以太网 UDP 通讯

4.5.1 传感器信息

数据输出速率: 可配置 100Hz/500Hz/1000Hz

IP 地址: 192.168.1.20

UDP 端口: 49152

传感器: Server

传感器加电启动后, 会持续监听 49152 端口的 UDP 输入请求, 执行相应的功能, 并做出响应。

4.5.2 通信协议

1、请求命令描述

命令	代码	功能描述	输出
停止	0x0000	停止数据输出	无
获取数据	0x0001 或 0x0002	根据请求命令中的计数 count 输出一组 (count = 1 时)、多组(count > 1 时)或持续输出(count=0 时) 响应包数据。	响应包数据
设置零点	0x0042	设置软件零点	无

2、请求数据包结构说明

```
typedef struct _req_struct
```

```
{  
    unsigned short magic;        // 固定为 0x1234  
    unsigned short command;      // 要执行的命令，如上所述  
    unsigned int count;          // 数据计数，为 0 时表示持续输出  
} reqStruct;
```

其中:

magic 占 2 字节，固定为 0x1234，用于识别请求命令。

command 占 2 字节，命令字，指定功能。具体定义见表 1-1。

count 占 4 字节，数据计数，用来指定该命令要输出的响应包个数。特别地，0 表示不限制个数，力传感器将持续输出响应包，直到收到停止命令。

注意：请求数据包中的数据字节要求是网络序。同样地，相应数据包中的数据字节也是网络序，需要先转换为本机字节序使用。

上位机通过 UDP 协议往 192.168.1.20 的 49152 端口发送如上所述的请求数据包，然后接收（或持续接收）力传感器返回的响应包数据。

3、响应数据包结构说明

```
typedef struct _resp_struct
```

```
{  
    unsigned int rdt_sequence;    // 响应数据包计数  
    unsigned int ft_sequence;     // 采集数据包计数  
    unsigned int status;         // 力传感器状态字  
    int Fx;                      // x 方向的力，除以 1000000 后单位为 N  
    int Fy;                      // y 方向的力，除以 1000000 后单位为 N  
    int Fz;                      // z 方向的力，除以 1000000 后单位为 N  
    int Tx;                      // x 向轴的力矩，除以 1000000 后单位为 N·m  
    int Ty;                      // y 向轴的力矩，除以 1000000 后单位为 N·m  
    int Tz;                      // z 向轴的力矩，除以 1000000 后单位为 N·m  
} respStruct;
```

其中:

rdt_sequence 占 4 字节，低字节在前，表示响应包数据序号。力传感器每发出一个响应包，rdt_sequence 都加 1。当力传感器收到新的请求包时，会将 rdt_sequence 重新设置为 1。也就是说，rdt_sequence 表示一次数据请求时响应数据包的序号，每次递增。上位机可以据

此判断是否有数据包丢失。

ft_sequence 占 4 字节，低字节在前，表示数据采集包序号。力传感器每采集一组数据，ft_sequence 都加 1。当 ft_sequence 的值为 0xFFFFFFFF 时，加 1 后会变为 0，然后继续累加。

Fx,Fy,Fz 各占 4 字节，低字节在前，分别表示 x, y, z 方向上除以 1000000 后以 N 为单位的力的数值。

Tx,Ty,Tz 各占 4 字节，低字节在前，分别表示以 x, y, z 方向为轴除以 1000000 后以 N·m 为单位的力矩值。

4.6 Modbus TCP 通讯

4.6.1 配置信息

IP 地址：192.168.0.20

端口号：502

传感器：Server(Slave)

数据发送频率：1000Hz

4.6.2 功能说明

传感器通电启动后，会持续监听 502 端口的 Modbus TCP 输入请求，执行相应的功能，并做出响应。

4.6.3 通信协议

1、指令描述

指令	代码	寄存器编号	功能描述	返回
读保持寄存器	0x03	0x0000	读力传感器力、力矩数据，长度 12。	力、力矩数据包
		0x1088	读传感器系统信息，长度 8。	传感器信息数据包
写单个寄存器	0x06	0x1000	设置零点：往编号为 0x1000 的寄存器写入值 1。	请求包
		0x1078-0x107F	分次写传感器系统信息。从 0x1078 到 0x107F，共 8 个寄存器，16 字节。每个寄存器 2 字节，8 个寄存器分别写入。	请求包
写多个寄存器	0x10	0x1078	一次性写传感器信息，长度 8。	回应包

说明：

1. 编号 0x1078 到 0x107F 的寄存器共 8 个，16 字节。用来存放输入的传感器系统信息；
2. 编号 0x1088 到 0x108F 的寄存器共 8 个，16 字节。用来存放输出的传感器系统信息；
3. 传感器系统信息需要先写入，再读出。即先依次写入 0x1078 到 0x107F，然后再读取 0x1088 开始的 8 个寄存器；
4. 传感器系统信息的写入可以通过多次调用写单个寄存器指令(0x06)或一次调用写多

个寄存器指令(0x10)完成。建议使用写多个寄存器指令；

5. 写多个寄存器返回的回应包参见“2. 写入传感器系统信息(0x10)”的返回。

6. 通过上述指令得到的力/力矩数据都是 16 位无符号整数，低字节在前，需先转换为十进制数据，再经过如下计算才能得到实际的力/力矩值：

实际力值(N) = (十进制数据 - 32768) / 32.768

实际力矩值(N·m) = (十进制数据 - 32768) / 327.68

2、指令范例

1. 写入传感器系统信息(0x06)

发送: 00 00 00 00 00 06 00 06 10 78 0F B0

返回: 00 00 00 00 00 06 00 06 10 78 0F B0

发送: 00 00 00 00 00 06 00 06 10 79 3F 10

返回: 00 00 00 00 00 06 00 06 10 79 3F 10

发送: 00 00 00 00 00 06 00 06 10 7A D9 40

返回: 00 00 00 00 00 06 00 06 10 7A D9 40

发送: 00 00 00 00 00 06 00 06 10 7B A3 01

返回: 00 00 00 00 00 06 00 06 10 7B A3 01

发送: 00 00 00 00 00 06 00 06 10 7C 7C 68

返回: 00 00 00 00 00 06 00 06 10 7C 7C 68

发送: 00 00 00 00 00 06 00 06 10 7D 34 BA

返回: 00 00 00 00 00 06 00 06 10 7D 34 BA

发送: 00 00 00 00 00 06 00 06 10 7E 49 CE

返回: 00 00 00 00 00 06 00 06 10 7E 49 CE

发送: 00 00 00 00 00 06 00 06 10 7F E8 2F

返回: 00 00 00 00 00 06 00 06 10 7F E8 2F

2. 写入传感器系统信息(0x10)

发送: 00 00 00 00 00 16 00 10 10 78 00 08 0F B0 3F 10 D9 40 A3 01 7C 68 34 BA 49 CE E8 2F

返回: 00 00 00 00 00 06 00 10 10 78 00 08

3. 读取传感器系统信息(0x03)

发送: 00 00 00 00 00 06 00 03 10 88 00 08

返回: 00 00 00 00 00 13 00 03 10 93 3F 7B AC 8C 19 6F 6D 06 5F CA A6 70 3A 16 3D

4. 设置零点(0x06)

发送: 00 00 00 00 00 06 00 06 10 00 00 01

返回: 00 00 00 00 00 06 00 06 10 00 00 01

5. 读取传感器力、力矩数据(0x03)

发送: 00 00 00 00 00 06 00 03 00 00 00 0C

返回: 00 00 00 00 00 1B 00 03 18 E6 3C 00 00 FB BB 00 00 F5 FE 00 00 00 1D 00 00 02 2E 00 00 00 36 00 00

第五章 维护

5.1 定期检修

对于需要经常移动系统电缆的工业型场景应用，用户应该定期检查电缆外层护套是否有磨损的迹象。WRIST 系列产品防护等级为 IP65，产品因避免被高压水流直接喷射和浸泡，应防止碎片和灰尘在传感器表面或传感器内部积累。如果受到环境污染，传感器表面可以用异丙醇清洗。如果在规定的量程范围内使用传感器，并使用适当的力矩规格安装拆卸，传感器本身应该没有损耗。

5.2 定期校准

对于学校、研究院等对产品精度要求极高的应用场所，用户需要定期返厂校准传感器。对于产品使用环境较为恶劣的领域，为保证产品的精度，我司建议用户每年至少校准一次。

5.3 故障诊断

本小节提示了设备可能出现的故障。对于手册中没有提到的故障或问题，用户可拨打我司技术支持电话寻求帮助。

传感器最常见的故障就是力和力矩读数错误，来自传感器的收到外界环境干扰时可能导致力和力矩读数的偏差和抖动。这些现象会导致传感器的精度问题。

造成传感器数据不准确的常见原因如下：

问题	原因/解决方案
噪声	传感器空载时力和力矩的读数波动超过量程的 0.1%是不正常的。信号扰动的原因可能是机械振动和电扰动，也有可能是地面不平整。噪声也可能象征着系统内的组件故障。 确保传感器的直流供电电压稳定。检测传感器线缆屏蔽层是否正确接地。
漂移	当一个负载卸载或再次施加时，数据的零点并不是稳定的，而是继续增加或减少。在使用偏差命令的分析时，可以更容易地观察到零点的变化。很大一部分是温度变化或机械耦合引起的漂移。 小范围的机械耦合造成的偏移是非常常见的，可以通过用户的程序滤波或者设置死区解决。
迟滞	当传感器被加载和卸载时，仪表读数不会迅速的响应输出稳定的读数。这样的滞后一般是由机械耦合或传感器内部特性引起的。小范围的迟滞是正常的。
无输出信号	确认传感器正确安装，工具侧和固定侧安装正确。检查供电。

第六章 产品规格

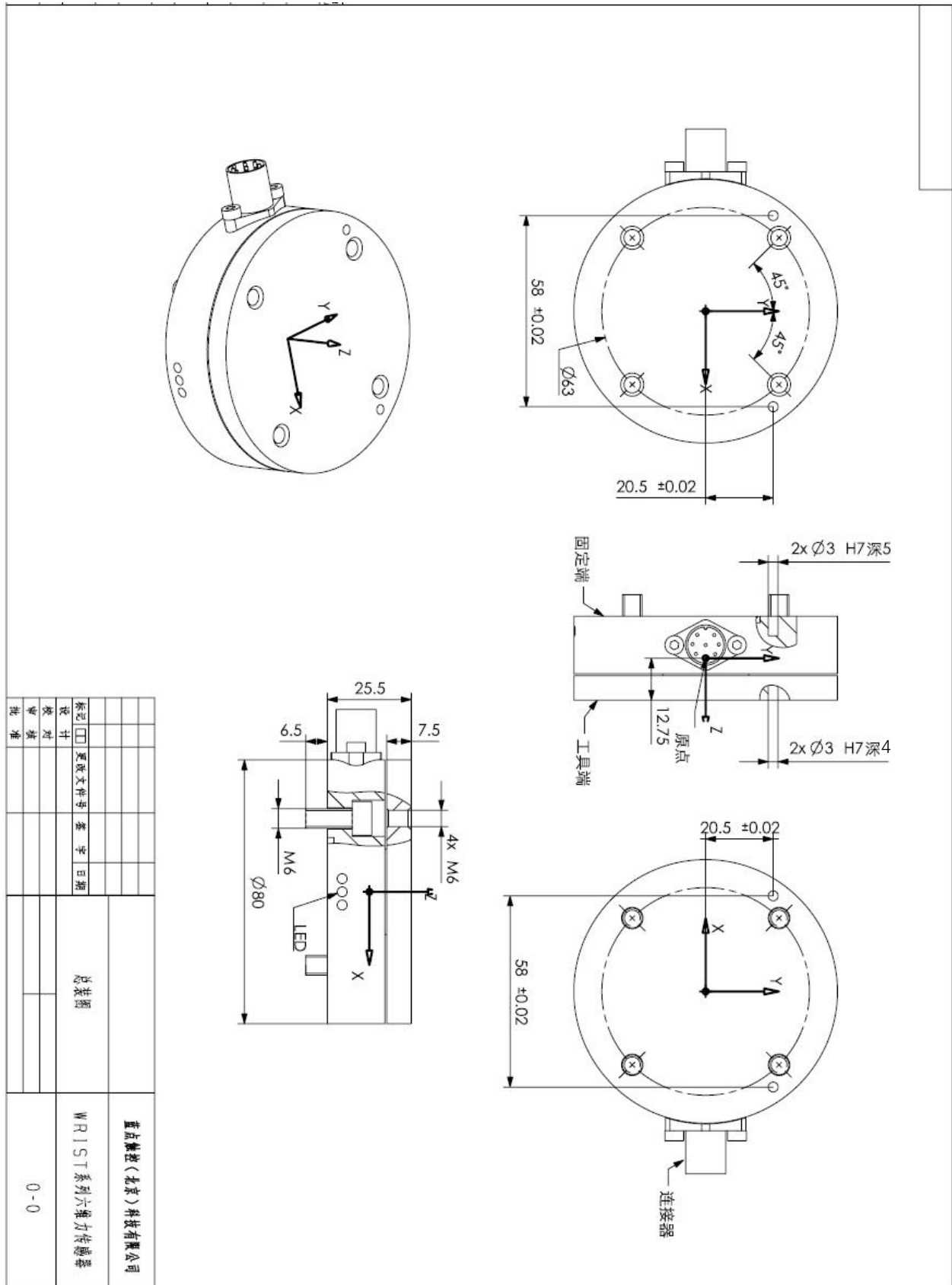
6.1 存储和使用条件

环境条件	
类型	定义
存储温度（摄氏度）	-20~85
使用温度（摄氏度）	-10~60
相对湿度	<95%，无水滴凝结
运输振动	5g
使用振动	2g
腐蚀性液体/气体	无防护

6.2 产品型号参数

六维力传感器		型号			
		ST-6-170-6	ST-6-200-10	ST-6-500-20	ST-6-1500-40
额定负载	F _x 、F _y (N)	170	200	500	1500
	F _z (N)	350	400	900	3000
	M _x 、M _y 、M _z (Nm)	6	10	20	40
分辨率	F _x 、F _y (N)	0.1	0.1	0.2	0.5
	F _z (N)	0.2	0.2	0.3	0.8
	M _x 、M _y 、M _z (Nm)	0.005	0.005	0.007	0.021
综合精度	含（滞后、线性、耦合、零飘）	1.2%FS	1.2%FS	1.2%FS	1.2%FS
过载		5 倍	5 倍	5 倍	5 倍
尺寸	mm	φ 80×H25.5	φ 80×H25.5	φ 80×H25.5	φ 80×H25.5
采样率	Hz	1000	1000	1000	1000
通讯		RS422/485 EtherCAT Ethernet	RS422/485 EtherCAT Ethernet	RS422/485 EtherCAT Ethernet	RS422/485 EtherCAT Ethernet
颜色		浅蓝	浅蓝	浅蓝	浅蓝
质量	g	约 260	约 260	约 280	约 620
防护等级		IP65/IP67	IP65/IP67	IP65/IP67	IP65/IP67
供电电压	V	DC 9-36	DC 9-36	DC 9-36	DC 9-36
温度补偿		内置	内置	内置	内置
功率	W	≤2	≤2	≤2	≤2

第七章 图纸



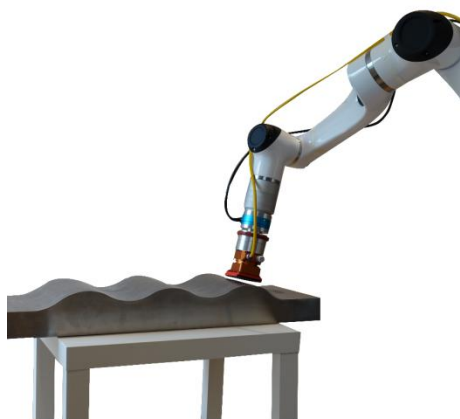
第八章 产品应用示例



示例一 磨边倒角



示例二 钻孔切削



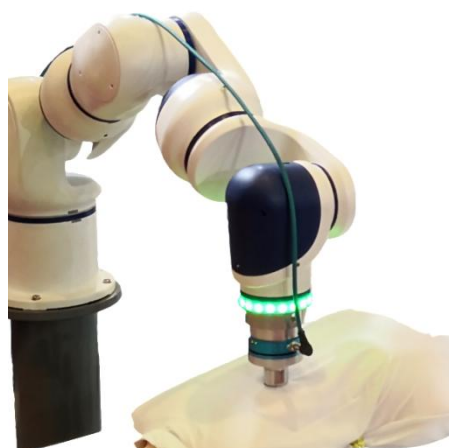
示例三 打磨抛光



示例四 柔性装配



示例五 碰撞检测



示例六 医疗按摩

附录 1 RS422 通讯示例代码

1、C++代码实现

以下代码仅处理数据包中第 2~33 字节的 float 解析。

```
float* result_floats = new float[8];  
memcpy(result_floats, data_char + 1, 32);
```

其中 data_char 为读取的字节

附录 2 以太网 UDP 通讯示例代码

1、Visual Studio 例程

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>

#pragma warning(disable: 4996)
#pragma comment(lib, "ws2_32")

#define PORT    49152    /* UDP 端口 */
#define COMMAND 2        /* 命令代码 */
#define SAMPLES 1        /* 响应包个数 */

typedef unsigned int uint32;
typedef int int32;
typedef unsigned short uint16;
typedef short int16;
typedef unsigned char byte;
typedef struct _resp_struct
{
    uint32 rdt_sequence;
    uint32 ft_sequence;
    uint32 status;
    int32 FTData[6];
} respStruct;

int main(int argc, char *argv[])
{
    WSADATA wsa_data;
    SOCKET sock;
    int str_len, len;
    SOCKADDR_IN serv_addr;
    struct sockaddr_in src;
    byte request[8];          /* 请求包数据 */
    respStruct resp;          /* 响应包结构 */
    byte respStruct[36];      /* 用于接收响应包数据的空间 */
    char * AXES[] = { "Fx", "Fy", "Fz", "Tx", "Ty", "Tz" };
    int i;

    if (argc != 3)
```

```

{
    printf("Usage: %s <IP> <port>\n", argv[0]);
    exit(1);
}
if (WSAStartup(MAKEWORD(2, 2), &wsa_data) != 0)
{
    printf("WSAStartup() error!\n");
    return 1;
}
sock = socket(AF_INET, SOCK_DGRAM, 0);
if (sock == INVALID_SOCKET)
{
    WSACleanup();
    printf("UDP socket create error\n");
    return 2;
}
memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = inet_addr(argv[1]);
serv_addr.sin_port = htons(atoi(argv[2]));
while (1)
{
    *(uint16*)&request[0] = htons(0x1234);
    *(uint16*)&request[2] = htons(COMMAND);
    *(uint32*)&request[4] = htonl(SAMPLES);
    len = sizeof(serv_addr);
    sendto(sock, request, 8, 0, (const struct sockaddr *) &serv_addr, len);
    str_len = recvfrom(sock, respStruct, sizeof(respStruct), 0, (struct sockaddr*)&src,
&len);
    if (str_len >= sizeof(respStruct))
    {
        resp.rdt_sequence = ntohl(*(uint32*)&respStruct[0]);
        resp.ft_sequence = ntohl(*(uint32*)&respStruct[4]);
        resp.status = ntohl(*(uint32*)&respStruct[8]);
        for (i = 0; i < 6; i++) {
            resp.FTData[i] = ntohl(*(int32*)&respStruct[12 + i * 4]);
        }
        printf("rdt, ft, status: 0x%08x, 0x%08x, 0x%08x\n", resp.rdt_sequence,
resp.ft_sequence, resp.status);
        for (i = 0; i < 6; i++) {
            printf("%s: %.5f\n", AXES[i], resp.FTData[i]/1000000.0);
        }
    }
}
}

```

```

    closesocket(sock);
    WSACleanup();
    return 0;
}

```

2、Linux 例程

2.1 Linux 下的历程编译

先确保安装好 gcc，在 shell 下执行如下编译命令：

```
$ gcc -o udpclient udpclient.c
```

如不出问题，当前目录下回生成可执行文件 udpclient。

2.2 例程运行

在 udpclient 所在目录执行如下命令：

```
$ ./udpclient 192.168.1.20 49152
```

如果力传感器正常启动，且和电脑网络能正常连接，则会显示如下结果：

```
rdt, ft, status: 0x00000001, 0x00007659, 0x00000000
```

```
Fx: -0.06721
```

```
Fy: -0.00109
```

```
Fz: -0.02455
```

```
Tx: 0.00027
```

```
Ty: 0.00552
```

```
Tz: 0.00049
```

```
rdt, ft, status: 0x00000001, 0x0000765a, 0x00000000
```

```
Fx: -0.06803
```

```
Fy: -0.01181
```

```
Fz: -0.02469
```

```
Tx: 0.00200
```

```
Ty: 0.00543
```

```
Tz: 0.00051
```

```
rdt, ft, status: 0x00000001, 0x0000765b, 0x00000000
```

```
Fx: -0.00685
```

```
Fy: -0.01183
```

```
Fz: -0.02581
```

```
Tx: 0.00027
```

```
Ty: 0.00054
```

```
Tz: 0.00059
```

2.3 udpclient.c 文件

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <string.h>
```

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
#include <netinet/in.h>
```

```
#include <arpa/inet.h>
```

```

#define PORT      49152  /* UDP 端口 */
#define COMMAND    2    /* 命令代码 */
#define NUM_SAMPLES 1    /* 响应包个数 */

typedef unsigned int uint32;
typedef int int32;
typedef unsigned short uint16;
typedef short int16;
typedef unsigned char byte;

typedef struct _resp_struct
{
    uint32 rdt_sequence;
    uint32 ft_sequence;
    uint32 status;
    int32 FTData[6];
} respStruct;

int main(int argc, char *argv[])
{
    int str_len, len;
    struct sockaddr_in serv_addr;
    struct sockaddr_in src;
    byte request[8];      /* 请求包数据 */
    respStruct resp;      /* 响应包结构 */
    byte respStruct[36];  /* 用于接收响应包数据的空间 */
    char *AXES[] = { "Fx", "Fy", "Fz", "Tx", "Ty", "Tz" };
    int i, sock;

    if (argc != 3)
    {
        printf("Usage: %s <IP> <port>\n", argv[0]);
        return -1;
    }
    sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0)
    {
        printf("UDP socket create error\n");
        return -2;
    }
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_addr.s_addr = inet_addr(argv[1]);

```

```

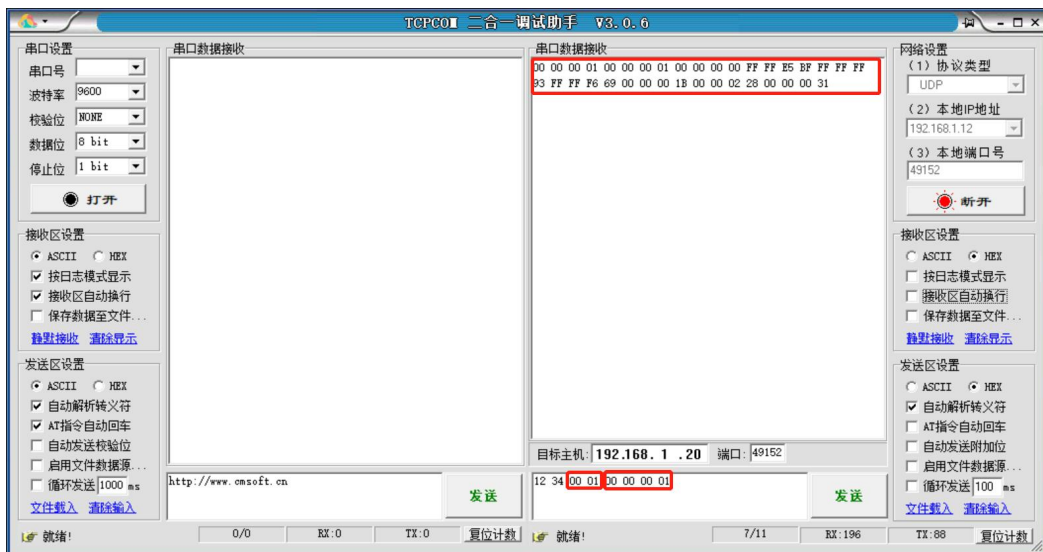
serv_addr.sin_port = htons(atoi(argv[2]));
while (1)
{
    *(uint16*)&request[0] = htons(0x1234);
    *(uint16*)&request[2] = htons(COMMAND);
    *(uint32*)&request[4] = htonl(NUM_SAMPLES);
    len = sizeof(serv_addr);
    sendto(sock, request, 8, 0, (const struct sockaddr *) &serv_addr, len);
    str_len = recvfrom(sock, respStruct, sizeof(respStruct), 0, (struct sockaddr *)
        &src, &len);
    if (str_len >= sizeof(respStruct))
    {
        resp.rdt_sequence = ntohl(*(uint32*)&respStruct[0]);
        resp.ft_sequence = ntohl(*(uint32*)&respStruct[4]);
        resp.status = ntohl(*(uint32*)&respStruct[8]);
        for (i = 0; i < 6; i++)
        {
            resp.FTDData[i] = ntohl(*(int32*)&respStruct[12 + i * 4]);
        }
        printf("rdt, ft, status: 0x%08x, 0x%08x, 0x%08x\n", resp.rdt_sequence,
            resp.ft_sequence, resp.status);
        for (i = 0; i < 6; i++)
        {
            printf("%s: %.5f\n", AXES[i], resp.FTDData[i]/1000000.0);
        }
    }
}
close(sock);
return 0;
}

```

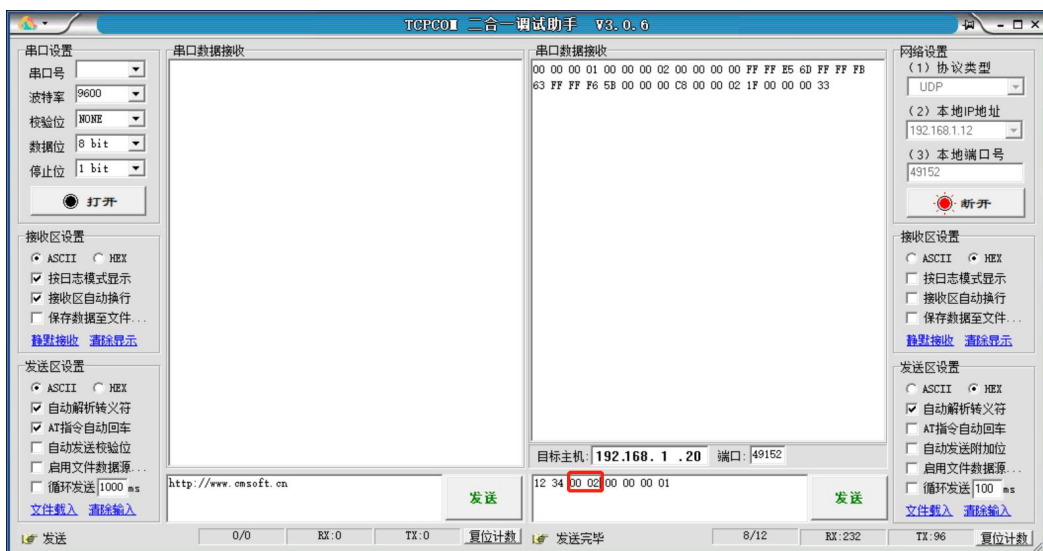
3、网络调试助手用例

3.1 发送获取数据命令 0x0001, 响应包个数 1

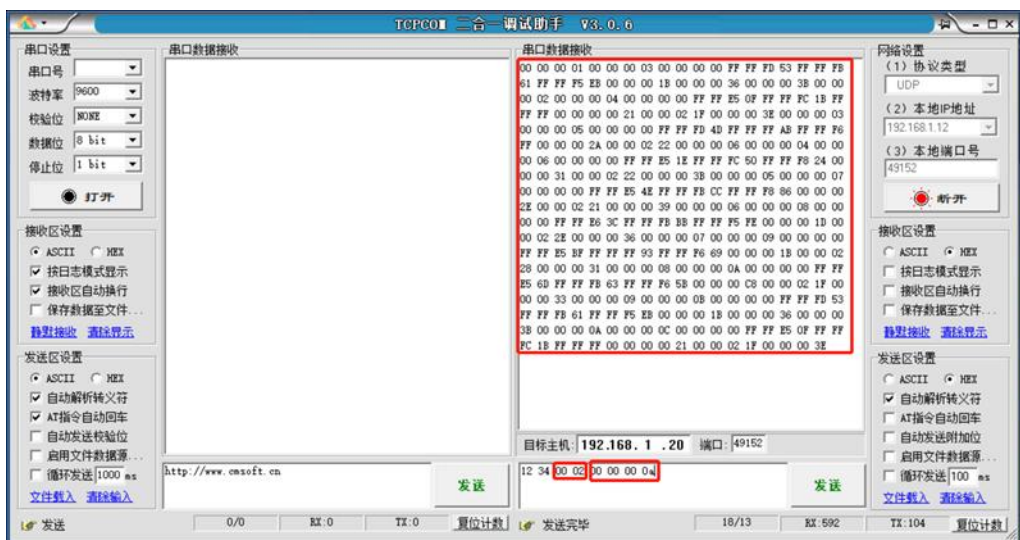
发送数据: 12 34 00 01 00 00 00 01



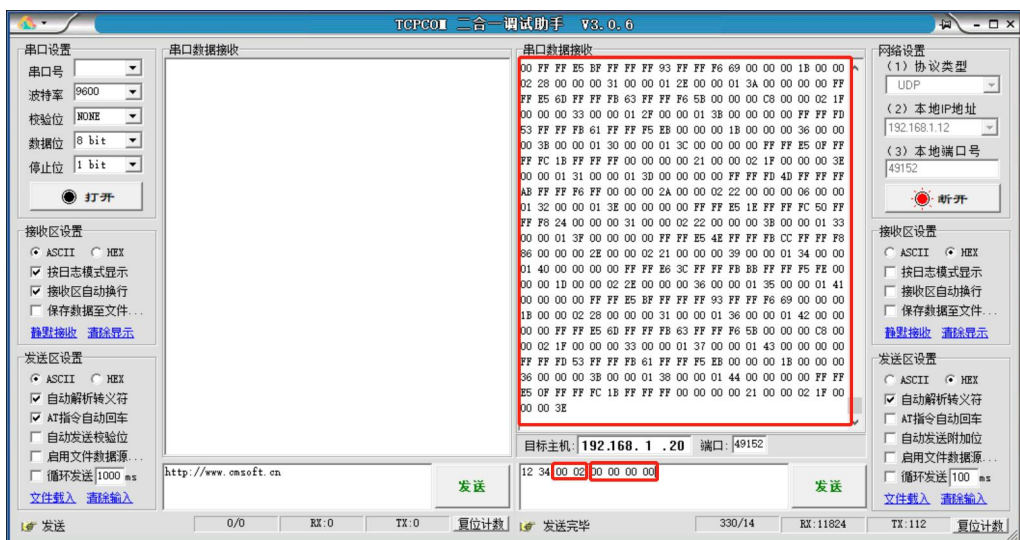
3.2 发送获取数据命令 0x0002, 响应包个数 1
发送数据: 12 34 00 02 00 00 00 01



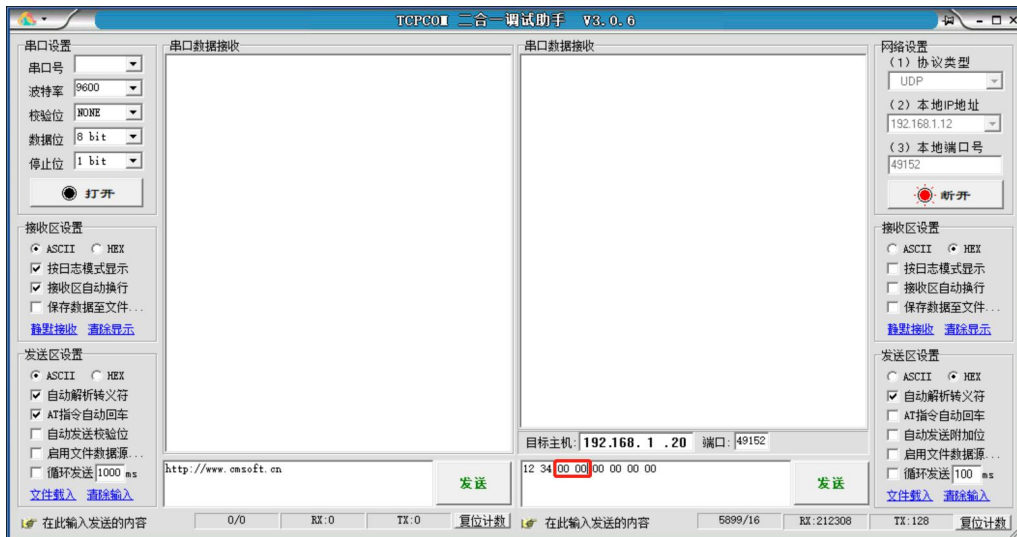
3.3 发送获取数据命令 0x0002, 响应包个数 10
发送数据: 12 34 00 02 00 00 00 0a



3.4 发送获取数据命令 0x0002, 响应包个数 0
发送数据: 12 34 00 02 00 00 00 00

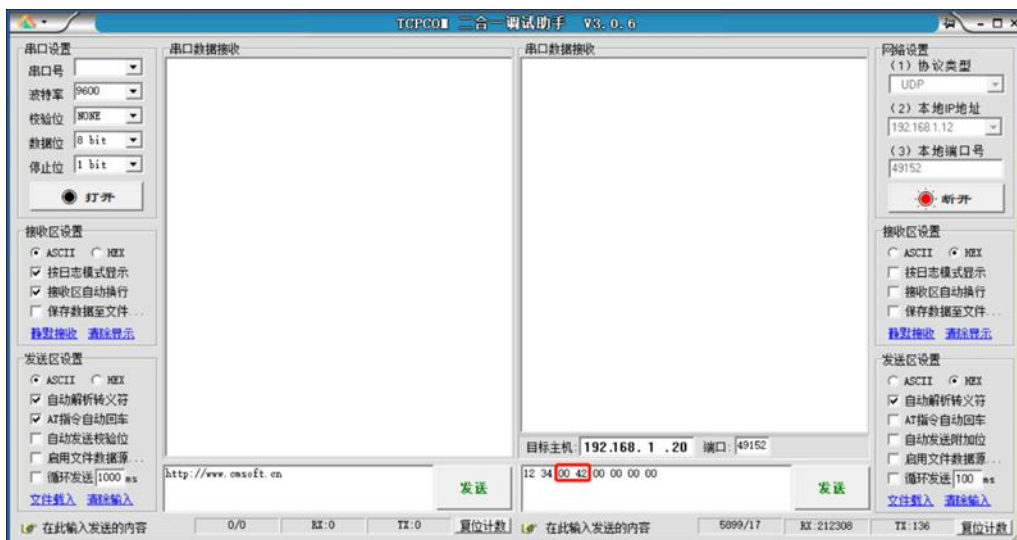


3.5 发送停止命令 0x0000
发送数据: 12 34 00 00 00 00 00 00



3.6 发送设置零点命令 0x0042

发送数据: 12 34 00 42 00 00 00 00



附录 3 Modbus TCP 通讯示例代码

1、传感器系统信息写入

控制器程序调用函数 `Inkt_open(&nDesc0,&nDesc1,&nDesc2,&nDesc3)`后，会得到系统返回的 4 个 4 字节长度的数值。例如，函数调用返回后，`nDesc0-nDesc3` 得到如下值：

```
nDesc0 = 0x103FB00F;  
nDesc1 = 0x01A340D9;  
nDesc2 = 0xBA34687C;  
nDesc3 = 0x2FE8CE49;
```

控制器程序调用指令 16(0x10)序列如下：

```
send_buf[0] = 0;  
send_buf[1] = 0;  
send_buf[2] = 0;  
send_buf[3] = 0;  
send_buf[4] = 0;  
send_buf[5] = 1+1+2+2+16;  
send_buf[6] = 0;  
send_buf[7] = 16;  
send_buf[8] = 0x10;  
send_buf[9] = 0x78;  
send_buf[10] = 0x00;  
send_buf[11] = 0x08;  
send_buf[12] = (nDesc0 >> 0) & 0xFF;  
send_buf[13] = (nDesc0 >> 8) & 0xFF;  
send_buf[14] = (nDesc0 >> 16) & 0xFF;  
send_buf[15] = (nDesc0 >> 24) & 0xFF;  
send_buf[16] = (nDesc1 >> 0) & 0xFF;  
send_buf[17] = (nDesc1 >> 8) & 0xFF;  
send_buf[18] = (nDesc1 >> 16) & 0xFF;  
send_buf[19] = (nDesc1 >> 24) & 0xFF;  
send_buf[20] = (nDesc2 >> 0) & 0xFF;  
send_buf[21] = (nDesc2 >> 8) & 0xFF;  
send_buf[22] = (nDesc2 >> 16) & 0xFF;  
send_buf[23] = (nDesc2 >> 24) & 0xFF;  
send_buf[24] = (nDesc3 >> 0) & 0xFF;  
send_buf[25] = (nDesc3 >> 8) & 0xFF;  
send_buf[26] = (nDesc3 >> 16) & 0xFF;  
send_buf[27] = (nDesc3 >> 24) & 0xFF;  
modbus_tcp_send(sClient,send_buf,28);
```

2、传感器系统信息读取

控制器程序在调用函数 `Inkt_start(pLnkT,nDesc0,nDesc1,nDesc2,nDesc3)` 之前，需要先调用指令 3 读取传感器系统信息，然后将传感器系统信息转换到 `nDesc0-nDesc3` 中。

例如：

```
send_buf[0] = 0;
send_buf[1] = 0;
send_buf[2] = 0;
send_buf[3] = 0;
send_buf[4] = 0;
send_buf[5] = 1+1+2+2;
send_buf[6] = 0;
send_buf[7] = 3;
send_buf[8] = 0x10;
send_buf[9] = 0x88;
send_buf[10] = 0x00;
send_buf[11] = 0x08;
modbus_tcp_send(sClient,send_buf,28);
cbRead = modbus_tcp_rcv(sClient,recv_buf,sizeof(recv_buf));
nDesc0 = (recv_buf[9] << 0) | (recv_buf[10] << 8) | (recv_buf[11] << 16) | (recv_buf[12]
<< 24);
nDesc1 = (recv_buf[13] << 0) | (recv_buf[14] << 8) | (recv_buf[15] << 16) | (recv_buf[16]
<< 24);
nDesc2 = (recv_buf[17] << 0) | (recv_buf[18] << 8) | (recv_buf[19] << 16) | (recv_buf[20]
<< 24);
nDesc3 = (recv_buf[21] << 0) | (recv_buf[22] << 8) | (recv_buf[23] << 16) | (recv_buf[24]
<< 24);
```

如果读取的系统信息为：

00 00 00 00 00 13 00 03 10 93 3F 7B AC 8C 19 6F 6D 06 5F CA A6 70 3A 16 3D

则 `nDesc0-nDesc3` 的值如下：

`nDesc0: 0xAC7B3F93`

`nDesc1: 0x6D6F198C`

`nDesc2: 0xA6CA5F06`

`nDesc3: 0x3D163A70`

更改记录

时间	版本	更改内容	更改人
2020.07.06	1.5.2	1、更新 4.6 Modbus TCP 通讯 2、新增附录 3 Modbus TCP 通讯示例代码	付伟宁
2020.08.19	1.5.3	1、更新首页产品图片 2、删除 3.5 中关于线色的描述	付伟宁
2020.08.26	1.5.4	1、新增 4.5.1 中传感器为 server 2、更新 4.5.2 中关于单位的描述 3、更新 6.2 产品型号参数	付伟宁
2020.12.02	1.5.5	1、更新电话号，新增网址 2、更新 4.2 中 L/A 状态灯的描述 3、更新 4.5 和 4.6 中大小端的描述 4、更新第七章图纸的销钉孔深度	付伟宁